

Introducción



El presente documento reúne una serie de diez algoritmos desarrollados en el lenguaje de programación Python, enfocados en los procesos de ordenamiento, búsqueda y análisis de rendimiento aplicados al manejo de datos.

Cada algoritmo representa una herramienta fundamental dentro del área de Estructura de Datos, permitiendo comprender la forma en que los sistemas procesan, organizan y optimizan grandes volúmenes de información.

A lo largo del contenido se presentan diferentes métodos de ordenamiento como Rápido, Burbuja, Selección, Inserción y Mezcla, cada uno acompañado de su respectivo pseudocódigo para facilitar la comprensión de su funcionamiento lógico.

Asimismo, se incluyen comparaciones de rendimiento entre algoritmos, ejercicios de búsqueda binaria y combinaciones de métodos aplicados a contextos prácticos como la clasificación de datos, la recomendación de productos y el análisis de eficiencia computacional.

Estos ejemplos permiten apreciar cómo los principios teóricos de los algoritmos se aplican en tareas reales dentro de la **Inteligencia Artificial**, la ciencia de datos y la automatización de procesos. En conjunto, el documento busca no solo ilustrar el funcionamiento técnico de cada algoritmo, sino también fomentar la comprensión crítica de su relevancia en la construcción de soluciones inteligentes y eficientes en el ámbito de la programación.

Ordenamiento de Burbuja: Clasificación de Muestras de Spam

Problema:

Un filtro de spam asigna una puntuación de riesgo a los correos electrónicos. Utiliza el algoritmo de burbuja para clasificarlos de menor a mayor riesgo.

Objetivo:

Comprender la lógica de comparación y cómo los algoritmos de ordenamiento pueden usarse en el pre-procesamiento de datos para la clasificación.

Pseudocódigo:

INICIO

correos $\leftarrow$ [0.9, 0.2, 0.5, 0.8, 0.1]

PARA i DESDE 0 HASTA longitud(correos)-2 HACER

PARA j DESDE 0 HASTA longitud(correos)-i-2 HACER

SI correos[j] > correos[j+1] ENTONCES

INTERCAMBIAR correos[j], correos[j+1]

FIN SI

FIN PARA

FIN PARA

FIN

Código en Python:

```
correos = [0.9, 0.2, 0.5, 0.8, 0.1]
```

```
for i in range(len(correos)-1):
```

```
    for j in range(len(correos)-i-1):
```

```
        if correos[j] > correos[j+1]:
```

```
correos[j], correos[j+1] = correos[j+1], correos[j]
```

```
print(correos)
```

Ordenamiento por Selección: Selección de Características

Problema:

Dada una lista de características con su importancia, usa el algoritmo de selección para ordenarlas de mayor a menor.

Objetivo:

Entender cómo este algoritmo es útil para encontrar los elementos extremos (mínimos o máximos) de un conjunto de datos de forma repetida.

Pseudocódigo:

INICIO

caracteristicas $\leftarrow$ [0.45, 0.9, 0.12, 0.67]

PARA i DESDE 0 HASTA longitud(caracteristicas)-1 HACER

 max_idx $\leftarrow$ i

 PARA j DESDE i+1 HASTA longitud(caracteristicas)-1 HACER

 SI caracteristicas[j] > caracteristicas[max_idx] ENTONCES

 max_idx $\leftarrow$ j

 FIN SI

 FIN PARA

 INTERCAMBIAR caracteristicas[i], caracteristicas[max_idx]

FIN PARA

FIN

Código en Python:

```
caracteristicas = [0.45, 0.9, 0.12, 0.67]
```

```
for i in range(len(caracteristicas)):
```

```
    max_idx = i
```

```
for j in range(i+1, len(caracteristicas)):

    if caracteristicas[j] > caracteristicas[max_idx]:

        max_idx = j

    caracteristicas[i], caracteristicas[max_idx] = caracteristicas[max_idx], caracteristicas[i]

print(caracteristicas)
```

Ordenamiento por Inserción: Clasificación de Temperaturas

Problema:

Ordenar una lista de temperaturas registradas en orden ascendente.

Objetivo:

Aplicar el algoritmo en un contexto de datos en vivo, demostrando cómo mantiene un conjunto de datos ordenado de manera eficiente cuando se insertan nuevos elementos.

Pseudocódigo:

INICIO

temperaturas $\leftarrow$ [32, 30, 35, 31, 29]

PARA i DESDE 1 HASTA longitud(temperaturas)-1 HACER

 clave $\leftarrow$ temperaturas[i]

$j \leftarrow i - 1$

 MIENTRAS $j \geq 0$ Y temperaturas[j] > clave HACER

 temperaturas[$j+1$] $\leftarrow$ temperaturas[j]

$j \leftarrow j - 1$

 FIN MIENTRAS

 temperaturas[$j+1$] $\leftarrow$ clave

FIN PARA

FIN

Código en Python:

```
temperaturas = [32, 30, 35, 31, 29]
```

```
for i in range(1, len(temperaturas)):
```

```
    clave = temperaturas[i]
```

```
    j = i - 1
```

```
    while j >= 0 and temperaturas[j] > clave:
```

```
temperaturas[j + 1] = temperaturas[j]
```

```
j -= 1
```

```
temperaturas[j + 1] = clave
```

```
print(temperaturas)
```

Ordenamiento por Mezcla: Eficiencia de Productos

Problema:

Dividir y ordenar una lista de eficiencias de productos usando MergeSort.

Objetivo:

Practicar la lógica de "divide y vencerás" y comprender cómo se fusionan listas ordenadas de manera eficiente, lo cual es útil en sistemas distribuidos.

Pseudocódigo:

INICIO

SI longitud(lista) > 1 ENTONCES

 mitad $\leftarrow$ longitud(lista) / 2

 izquierda $\leftarrow$ lista[0:mitad]

 derecha $\leftarrow$ lista[mitad:]

 MergeSort(izquierda)

 MergeSort(derecha)

 combinar(izquierda, derecha, lista)

FIN SI

FIN

Código en Python:

```
def merge_sort(lista):
```

```
    if len(lista) > 1:
```

```
        mitad = len(lista)//2
```

```
        izquierda = lista[:mitad]
```

```
        derecha = lista[mitad:]
```



```
merge_sort(izquierda)
```

```
merge_sort(derecha)
```

```
i = j = k = 0
```

```
while i < len(izquierda) and j < len(derecha):
```

```
    if izquierda[i] < derecha[j]:
```

```
        lista[k] = izquierda[i]
```

```
        i += 1
```

```
    else:
```

```
        lista[k] = derecha[j]
```

```
        j += 1
```

```
    k += 1
```

```
while i < len(izquierda):
```

```
    lista[k] = izquierda[i]
```

```
    i += 1
```

```
    k += 1
```

```
while j < len(derecha):
```

```
    lista[k] = derecha[j]
```

```
    j += 1
```

```
    k += 1
```

```
productos = [45, 12, 78, 34, 23]
```

```
merge_sort(productos)
```

```
print(productos)
```

QuickSort: Clasificación de Puntuaciones IA

Problema:

Ordenar las puntuaciones de modelos de IA de menor a mayor usando QuickSort.

Objetivo:

Aplicar uno de los algoritmos más eficientes para ordenar grandes volúmenes de datos, un requisito crucial en aplicaciones de IA a escala.

Pseudocódigo:

INICIO

SI lista tiene más de un elemento ENTONCES

 pivote $\leftarrow$ último elemento

 menores $\leftarrow$ elementos $<$ pivote

 mayores $\leftarrow$ elementos $\geq$ pivote

 return QuickSort(menores) + [pivote] + QuickSort(mayores)

FIN SI

FIN

Código en Python:

```
def quicksort(lista):
```

```
    if len(lista) <= 1:
```

```
        return lista
```

```
    pivote = lista[-1]
```

```
    menores = [x for x in lista[:-1] if x < pivote]
```

```
    mayores = [x for x in lista[:-1] if x >= pivote]
```

```
    return quicksort(menores) + [pivote] + quicksort(mayores)
```

```
puntuaciones = [0.8, 0.4, 0.9, 0.3, 0.7]
```

```
print(quicksort(puntuaciones))
```

Análisis de Rendimiento: Burbuja vs. Inserción

Problema:

Implementa el algoritmo de **ordenamiento de burbuja** y el de **ordenamiento por inserción**. Mide el tiempo de ejecución de cada uno en arreglos de 100, 1,000 y 10,000 elementos.

Objetivo:

Comparar de forma empírica la diferencia de rendimiento entre un algoritmo de complejidad $O(n^2)$ para diferentes tamaños de datos.

Pseudocódigo:

Función burbuja:

Inicio función burbuja(lista)

$n \leftarrow \text{longitud}(\text{lista})$

 Para $i \leftarrow 0$ hasta $n - 2$ hacer

 Para $j \leftarrow 0$ hasta $n - i - 2$ hacer

 Si $\text{lista}[j] > \text{lista}[j + 1]$ entonces

 Intercambiar $\text{lista}[j]$ y $\text{lista}[j + 1]$

 Fin Si

 Fin Para

 Fin Para

 retornar lista

Fin función

Función inserción

Inicio función insertion_sort(lista)

 Para $i \leftarrow 1$ hasta $\text{longitud}(\text{lista}) - 1$ hacer

 actual $\leftarrow \text{lista}[i]$

$j \leftarrow i - 1$

 Mientras $j \geq 0$ y $\text{lista}[j] > \text{actual}$ hacer

$\text{lista}[j + 1] \leftarrow \text{lista}[j]$

$j \leftarrow j - 1$

Fin Mientras

lista[j + 1] ← actual

Fin Para

retornar lista

Fin función

Función principal

Inicio

tamaños ← [100, 1000, 3000]

Para cada tamaño en tamaños hacer

valores ← lista de números aleatorios entre 10 y 100000 con longitud = tamaño

lista_burbuja ← copia de valores

lista_insercion ← copia de valores

(_, tiempo_burbuja) ← medir_tiempo(burbuja, lista_burbuja)

(_, tiempo_insercion) ← medir_tiempo(insertion_sort, lista_insercion)

Mostrar "Tamaño de lista:", tamaño

Mostrar "Burbuja:", tiempo_burbuja, "segundos"

Mostrar "Inserción:", tiempo_insercion, "segundos"

Fin Para

Fin

Código en Python:

```
import time
```

```
import random
```

```
# Función para medir tiempos de ejecución
```

```
def medir_tiempo(func, *args):
```

```
    inicio = time.time()
```

```
    resultado = func(*args)
```

```
    final = time.time()
```

```
    return resultado, final - inicio
```

```
# Ordenamiento burbuja
```

```
def burbuja(lista):
```

```
    n = len(lista)
```

```
    for i in range(n - 1):
```

```
        for j in range(n - i - 1):
```

```
            if lista[j] > lista[j + 1]:
```

```
                # intercambiar los elementos
```

```
                lista[j], lista[j + 1] = lista[j + 1], lista[j]
```

```
    return lista
```

```
# Ordenamiento por inserción
```

```
def insertion_sort(lista):
```

```
    for i in range(1, len(lista)):
```

```
        actual = lista[i]
```

```
        j = i - 1
```

```
        while j >= 0 and lista[j] > actual:
```

```
            lista[j + 1] = lista[j]
```

```
            j -= 1
```

```
        lista[j + 1] = actual

    return lista

# Tamaños de prueba

sizes = [100, 1000, 10000]

# Comparación de tiempos

for size in sizes:

    valores = [random.randint(10, 100000) for _ in range(size)]

    # Copias de la misma lista para no alterar los datos originales

    lista_burbuja = valores.copy()

    lista_insercion = valores.copy()

    # Medir tiempos

    _, tiempo_burbuja = medir_tiempo(burbuja, lista_burbuja)

    _, tiempo_insercion = medir_tiempo(insertion_sort, lista_insercion)

    # Mostrar resultados

    print(f"\nTamaño de lista: {size}\n - Burbuja: {tiempo_burbuja:.6f} segundos\n- Inserción: {tiempo_insercion:.6f} segundos")
```

Análisis de Rendimiento: MergeSort vs. QuickSort

Problema:

Implementa los algoritmos de **MergeSort** y **QuickSort**. Mide y compara su tiempo de ejecución para ordenar arreglos de 1,000 y 10,000 elementos.

Objetivo:

Analizar las ventajas de los algoritmos de "divide y vencerás" sobre los algoritmos de ordenamiento simple y observar cómo manejan el rendimiento de los datos a gran escala.

Pseudocódigo:

Inicio

Definir función Medir_Tiempo(algoritmo, lista):

 tiempo_inicio $\leftarrow$ tiempo actual

 resultado $\leftarrow$ ejecutar algoritmo(lista)

 tiempo_final $\leftarrow$ tiempo actual

 retornar (resultado, tiempo_final - tiempo_inicio)

Definir función MergeSort(arreglo):

 Si longitud(arreglo) ≤ 1 entonces

 retornar arreglo

 FinSi

 mitad $\leftarrow$ longitud(arreglo) / 2

 izquierda $\leftarrow$ MergeSort(arreglo[0:mitad])

 derecha $\leftarrow$ MergeSort(arreglo[mitad:final])

 retornar Merge(izquierda, derecha)

Definir función Merge(izquierda, derecha):

 resultado $\leftarrow$ lista vacía

 i $\leftarrow 0$

 j $\leftarrow 0$

 Mientras i < longitud(izquierda) Y j < longitud(derecha):

Si $\text{izquierda}[i] < \text{derecha}[j]$ entonces

agregar $\text{izquierda}[i]$ a resultado

$i \leftarrow i + 1$

Sino

agregar $\text{derecha}[j]$ a resultado

$j \leftarrow j + 1$

FinSi

FinMientras

agregar los elementos restantes de izquierda y derecha a resultado

retornar resultado

Definir función $\text{Particion}(\text{lista}, \text{inicio}, \text{fin})$:

$\text{pivote} \leftarrow \text{lista}[\text{fin}]$

$i \leftarrow \text{inicio} - 1$

Para j desde inicio hasta $\text{fin} - 1$ hacer

Si $\text{lista}[j] \leq \text{pivote}$ entonces

$i \leftarrow i + 1$

intercambiar $\text{lista}[i]$ con $\text{lista}[j]$

FinSi

FinPara

intercambiar $\text{lista}[i + 1]$ con $\text{lista}[\text{fin}]$

retornar $i + 1$

Definir función $\text{QuickSort}(\text{lista}, \text{inicio}, \text{fin})$:

Si $\text{inicio} < \text{fin}$ entonces

$\text{pivote} \leftarrow \text{Particion}(\text{lista}, \text{inicio}, \text{fin})$

$\text{QuickSort}(\text{lista}, \text{inicio}, \text{pivote} - 1)$

$\text{QuickSort}(\text{lista}, \text{pivote} + 1, \text{fin})$

FinSi

tamaños $\leftarrow$ [100, 1000, 10000]

Para cada tamaño en tamaños hacer:

lista_original $\leftarrow$ generar lista aleatoria de tamaño 'tamaño'

lista_merge $\leftarrow$ copia de lista_original

lista_quick $\leftarrow$ copia de lista_original

(resultado1, tiempo_merge) $\leftarrow$ Medir_Tiempo(MergeSort, lista_merge)

(resultado2, tiempo_quick) $\leftarrow$ Medir_Tiempo(QuickSort, lista_quick)

Mostrar "Tamaño de lista:", tamaño

Mostrar "- Merge Sort:", tiempo_merge, "segundos"

Mostrar "- Quick Sort:", tiempo_quick, "segundos"

FinPara

Fin

Código en Python:

```
import time

import random

# Función para medir tiempos de ejecución

def medir_tiempo(func, *args):

    inicio = time.time()

    resultado = func(*args)

    final = time.time()

    return resultado, final - inicio

# ----- MERGE SORT -----

def mergeSort(arr):

    if len(arr) <= 1:

        return arr

    mid = len(arr) // 2

    leftHalf = arr[:mid]

    rightHalf = arr[mid:]

    sortedLeft = mergeSort(leftHalf)

    sortedRight = mergeSort(rightHalf)

    return merge(sortedLeft, sortedRight)

def merge(left, right):

    result = []

    i = j = 0

    while i < len(left) and j < len(right):

        if left[i] < right[j]:

            result.append(left[i])
```

```
i += 1
```

```
else:
```

```
    result.append(right[j])
```

```
    j += 1
```

```
result.extend(left[i:])
```

```
result.extend(right[j:])
```

```
return result
```

```
# ----- QUICK SORT -----
```

```
def particion(lista, inicio, fin):
```

```
    pivote = lista[fin]
```

```
    i = inicio - 1
```

```
    for j in range(inicio, fin):
```

```
        if lista[j] <= pivote:
```

```
            i += 1
```

```
            lista[i], lista[j] = lista[j], lista[i]
```

```
    lista[i + 1], lista[fin] = lista[fin], lista[i + 1]
```

```
    return i + 1
```

```
def quicksort(A, inicio, fin):
```

```
    if inicio < fin:
```

```
        indice_pivote = particion(A, inicio, fin)
```

```
        quicksort(A, inicio, indice_pivote - 1)
```

```
        quicksort(A, indice_pivote + 1, fin)
```

```
# ----- PRUEBA Y COMPARACIÓN -----
```

```
sizes = [100, 1000, 10000]
```

```
for size in sizes:
```

```
    valores = [random.randint(10, 100000) for _ in range(size)]
```

```
lista_merge = valores.copy()
```

```
lista_quick = valores.copy()
```

```
# Medir tiempos
```

```
_, tiempo_merge = medir_tiempo(mergeSort, lista_merge)
```

```
_, tiempo_quick = medir_tiempo(quicksort, lista_quick, 0, len(lista_quick) - 1)
```

```
# Mostrar resultados
```

```
print(f"\nTamaño de lista: {size}")
```

```
print(f"- Merge Sort: {tiempo_merge:.6f} segundos")
```

```
print(f"- Quick Sort: {tiempo_quick:.6f} segundos")
```

Búsqueda Binaria después de QuickSort

Problema:

Un conjunto de datos de 500 clientes está desordenado. Utiliza **QuickSort** para ordenar los datos y, una vez ordenados, implementa la **búsqueda binaria** para encontrar un cliente específico.

Objetivo:

Demostrar cómo los algoritmos de ordenamiento son un paso previo esencial para optimizar la eficiencia de otros algoritmos, como la búsqueda.

Pseudocódigo:

Inicio

Definir función Busqueda_Binaria(lista, objetivo):

low $\leftarrow$ 0

high $\leftarrow$ longitud(lista) - 1

Mientras low $\leq$ high hacer:

mid $\leftarrow$ (low + high) // 2

Si lista[mid] == objetivo entonces

retornar mid

Sino si lista[mid] < objetivo entonces

low $\leftarrow$ mid + 1

Sino

high $\leftarrow$ mid - 1

FinSi

FinMientras

retornar -1 // Elemento no encontrado

FinFunción

// Ejemplo de ejecución

lista $\leftarrow$ [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

objetivo $\leftarrow$ 23

posición $\leftarrow$ Busqueda_Binaria(lista, objetivo)

Si posición $\neq$ -1 entonces

Mostrar "El valor se encuentra en la posición:", posición

Sino

Mostrar "El valor no se encuentra en la lista"

FinSi

Fin

Código en Python:

```
def busqueda_binaria(lista, objetivo):
```

```
    low, high = 0, len(lista) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if lista[mid] == objetivo:
```

```
            return mid
```

```
        elif lista[mid] < objetivo:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1
```

```
# Ejemplo de uso
```

```
datos = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
```

```
valor_buscado = 23
```

```
resultado = busqueda_binaria(datos, valor_buscado)
```

```
if resultado != -1:
```

```
    print(f"El valor {valor_buscado} se encuentra en la posición {resultado}.")
```

```
else:
```

```
    print(f"El valor {valor_buscado} no se encuentra en la lista.")
```

Combinación de Algoritmos: Clasificación y Recomendación

Problema:

Crea un conjunto de datos de 100 productos con una "puntuación de relevancia". Utiliza **MergeSort** para ordenar la lista de productos y luego, implementa una lógica que muestre solo los 5 productos con mayor puntuación, simulando un sistema de recomendación.

Objetivo:

Combinar múltiples algoritmos para resolver un problema de la vida real en IA, mostrando cómo las diferentes partes del conocimiento se integran en una solución funcional.

Pseudocódigo:

Inicio

 Crear lista vacía llamada productos

 Para i desde 1 hasta 100 hacer:

 producto $\leftarrow$ {nombre: "Producto_" + i, puntuacion: número aleatorio entre 1 y 100}

 Agregar producto a productos

 FinPara

 Definir función MergeSort(arreglo):

 Si longitud(arreglo) $\leq$ 1 entonces

 retornar arreglo

 FinSi

 mitad $\leftarrow$ longitud(arreglo) / 2

 izquierda $\leftarrow$ MergeSort(arreglo[0:mitad])

 derecha $\leftarrow$ MergeSort(arreglo[mitad:final])

 retornar Merge(izquierda, derecha)

 FinFunción

 Definir función Merge(izquierda, derecha):

 resultado $\leftarrow$ lista vacía

 i $\leftarrow$ 0

$j \leftarrow 0$

Mientras $i < \text{longitud}(\text{izquierda})$ Y $j < \text{longitud}(\text{derecha})$ hacer:

Si $\text{izquierda}[i] < \text{derecha}[j]$ entonces

agregar $\text{izquierda}[i]$ a resultado

$i \leftarrow i + 1$

Sino

agregar $\text{derecha}[j]$ a resultado

$j \leftarrow j + 1$

FinSi

FinMientras

agregar los elementos restantes de izquierda y derecha a resultado

retornar resultado

FinFunción

$\text{puntuaciones} \leftarrow \text{extraer puntuacion de cada producto}$

$\text{puntuaciones_ordenadas} \leftarrow \text{MergeSort}(\text{puntuaciones})$

$\text{top_5_puntuaciones} \leftarrow \text{últimos 5 elementos de puntuaciones_ordenadas, en orden descendente}$

$\text{productos_recomendados} \leftarrow \text{productos cuyas puntuaciones están en top_5_puntuaciones}$

Mostrar "=== SISTEMA DE RECOMENDACIÓN DE PRODUCTOS ==="

Para cada producto en $\text{productos_recomendados}$ hacer:

Mostrar producto.nombre y $\text{producto.puntuacion}$

FinPara

Fin

Código de Python:

```
import random

# ----- CREACIÓN DE DATOS -----

# Generar 100 productos con puntuaciones aleatorias de relevancia

productos = []

for i in range(1, 101):

    producto = {

        "nombre": f"Producto_{i}",

        "puntuacion": random.randint(1, 100)

    }

    productos.append(producto)

# ----- MERGE SORT -----

def mergeSort(arr):

    if len(arr) <= 1:

        return arr

    mid = len(arr) // 2

    leftHalf = arr[:mid]

    rightHalf = arr[mid:]

    sortedLeft = mergeSort(leftHalf)

    sortedRight = mergeSort(rightHalf)

    return merge(sortedLeft, sortedRight)

def merge(left, right):

    result = []

    i = j = 0

    while i < len(left) and j < len(right):

        if left[i] < right[j]:
```

```

        result.append(left[i])

        i += 1

    else:

        result.append(right[j])

        j += 1

    result.extend(left[i:])

    result.extend(right[j:])

    return result

# ----- APLICACIÓN DEL ORDENAMIENTO -----

# Extraer las puntuaciones de los productos

puntuaciones = [p["puntuacion"] for p in productos]

# Ordenar las puntuaciones con MergeSort

puntuaciones_ordenadas = mergeSort(puntuaciones)

# Obtener las 5 puntuaciones más altas

top_5_puntuaciones = puntuaciones_ordenadas[-5:][::-1]

# Buscar los productos que tengan esas puntuaciones

productos_recomendados = []

for p in productos:

    if p["puntuacion"] in top_5_puntuaciones and p not in productos_recomendados:

        productos_recomendados.append(p)

    if len(productos_recomendados) == 5:

        break

# ----- MOSTRAR RECOMENDACIONES -----

print("=== SISTEMA DE RECOMENDACIÓN DE PRODUCTOS ===\n")

for prod in productos_recomendados:

    print(f'{prod["nombre"]} - Puntuación: {prod["puntuacion"]}')

```


Taller de Recursividad: Visualización de MergeSort

Problema:

El **ordenamiento por mezcla** es un algoritmo recursivo. Los estudiantes deben dibujar o visualizar de forma manual cómo se divide un arreglo de 8 elementos en subarreglos más pequeños y cómo se vuelven a mezclar, paso a paso, hasta que el arreglo completo esté ordenado.

Objetivo:

Reforzar la comprensión de la recursividad y la lógica de "divide y vencerás" de una manera visual y tangible.

Procedimiento:

Supongamos el siguiente arreglo de 8 elementos:

[38, 27, 43, 3, 9, 82, 10, 50]

- ☐ Paso 1: Dividir el arreglo en mitades
[38, 27, 43, 3] | [9, 82, 10, 50]
- ☐ Paso 2: Dividir cada mitad nuevamente
[38, 27] | [43, 3] | [9, 82] | [10, 50]
- ☐ Paso 3: Dividir hasta llegar a subarreglos de 1 elemento
[38] | [27] | [43] | [3] | [9] | [82] | [10] | [50]
En este punto, cada subarreglo tiene un solo elemento → se considera ordenado.
- ☐ Paso 4: Comenzar a combinar (fase de mezcla)
[38] y [27] → [27, 38]
[43] y [3] → [3, 43]
[9] y [82] → [9, 82]
[10] y [50] → [10, 50]
- ☐ Paso 5: Mezclar los resultados intermedios
[27, 38] y [3, 43] → [3, 27, 38, 43]
[9, 82] y [10, 50] → [9, 10, 50, 82]
- ☐ Paso 6: Mezcla final
[3, 27, 38, 43] y [9, 10, 50, 82]
→ [3, 9, 10, 27, 38, 43, 50, 82]

✓ Arreglo ordenado final:

[3, 9, 10, 27, 38, 43, 50, 82]

Conclusión



A lo largo del desarrollo de los distintos algoritmos de ordenamiento y análisis implementados en Python, fue posible comprender de manera práctica cómo cada técnica contribuye al tratamiento eficiente de datos en la Inteligencia Artificial y la programación.

El estudio de métodos como Burbuja, Selección e Inserción permitió visualizar la lógica secuencial de los procesos básicos, mientras que MergeSort y QuickSort demostraron la potencia de la recursividad y la eficiencia computacional.

Estos ejercicios fortalecieron el pensamiento lógico, la capacidad de abstracción y la comprensión del impacto que tienen los algoritmos en el procesamiento eficiente de datos.

Más allá de su aplicación técnica, estos conceptos son fundamentales para el desarrollo de soluciones inteligentes y la toma de decisiones automatizadas en el ámbito de la IA.